Spring Semester 2025-26

Final Project Report

DSCI-551 : FOUNDATIONS OF DATA MANAGEMENT

Design and Performance Analysis of a Flight Search System Using SQLite B-Tree Indexing

Prof. Wensheng Wu

Submitted by :

Abhey Sabesan Mageswaran Aryaan

7162-7286-71

1) Introduction & Motivation

Modern applications rely heavily on database systems to store, retrieve, and manage large volumes of data efficiently. As datasets grow into the millions or billions of rows, the performance of database queries becomes a critical factor in determining whether an application is usable or not. A query that takes half a second on a small dataset may take minutes on a production-scale dataset, making index design one of the most important skills in database engineering.

This project investigates how B-Tree indexing in SQLite impacts query performance at scale. To demonstrate this, a Flight Search System was designed and implemented, a simulated application that stores 1 million flight records and allows users to search for flights by route, filter by airline, and sort by price. The application was intentionally kept simple so that the emphasis could be placed entirely on measuring and proving the internal database mechanisms that drive performance.

The motivation for this project comes from a fundamental question in database design: how does the internal structure of a database index translate into real-world application performance? By measuring query times before and after index creation, and by using SQLite's internal tools, EXPLAIN QUERY PLAN and VDBE bytecode, this project provides concrete, measurable evidence of how B-Tree indexing works internally and why it matters for application behavior.

2) Database System Overview

SQLite is a lightweight, embedded, open-source relational database management system. Unlike traditional database systems such as MySQL or PostgreSQL, SQLite does not operate as a separate server process. Instead, it runs directly within the application and stores all data in a single file on disk. This embedded, serverless architecture makes SQLite the most widely deployed database engine in the world, used in mobile applications, browsers, operating systems, and embedded devices.

SQLite was selected for this project for three primary reasons.

1. It provides a well-documented and transparent internal architecture that exposes its B-Tree indexing and query planning mechanisms through tools like EXPLAIN QUERY PLAN and EXPLAIN (VDBE bytecode).
2. SQLite stores both tables and indexes using B-Tree data structures internally, making it particularly suitable for focused analysis of indexing behavior.
3. SQLite comes built into Python via the sqlite3 module, requiring no additional installation, which allowed the project to focus entirely on database analysis rather than infrastructure setup.

2.1 : Key Characteristics

- Embedded and serverless, which means no separate server process required
- All data stored in a single .db file on disk
- Full ACID compliance, i.e. Atomicity, Consistency, Isolation, Durability
- Built-in support for B-Tree indexing, query planning, and VDBE execution
- EXPLAIN QUERY PLAN and EXPLAIN commands expose internal execution strategies
- Native Python integration via sqlite3 module, so no installation needed

3) Internal Architecture

3.1 : Storage Architecture

SQLite organizes all data using B-Tree data structures. When a table is created, SQLite builds a B-Tree where each leaf node contains actual row records and internal nodes guide the search process. This hierarchical structure allows SQLite to locate specific records in O(log n) time rather than O(n) time — a critical distinction for large datasets.

When an index is created, SQLite builds a completely separate B-Tree structure. For a composite index on (origin, destination), the index B-Tree is sorted alphabetically by origin first, then by destination within each origin group. Each leaf node in the index B-Tree contains the indexed column values plus a pointer back to the corresponding row in the main table B-Tree. This separation of concerns allows SQLite to maintain multiple sorted views of the same data without duplicating the full row records.

3.2 : Indexing- Composite v/s Single Column

I analyzed two types of indexes: composite indexes covering multiple columns and single-column indexes. The composite index on (origin, destination) proved dramatically more effective than the single-column index on airline. This difference is explained by index selectivity, the degree to which an index narrows down the result set.

The airline column has only 5 distinct values (Delta, United, American, Southwest, JetBlue), meaning a single-column index on airline still requires fetching approximately 200,000 rows out of 1 million. In contrast, the composite index on (origin, destination) has 56 possible combinations across 8 airports, meaning a query for LAX to JFK fetches only approximately 18,000 rows, a much smaller and more specific result set. This difference in selectivity directly explains the performance gap: 11x speedup for the single-column index versus 90-100x speedup for the composite index.

3.3 : Query Execution – The Query Planner

When a SQL query is submitted, SQLite's query planner evaluates all available indexes and determines the most efficient execution strategy. This decision is based on estimated query cost , the planner compares the cost of a full table scan against the cost of navigating each available index. If the planner determines that an index will reduce the number of rows examined, it selects an index scan. If no suitable index exists, or if the available index does not match the query conditions, the planner falls back to a full table scan.

The EXPLAIN QUERY PLAN command exposes this decision, returning either SCAN flights (full table scan) or SEARCH flights USING INDEX idx_origin_destination (index scan). This project demonstrated that the planner is intelligent, in the wrong index experiment, a price index was created but a query filtered by origin and destination. The planner correctly recognized the price index was irrelevant for this query and chose a full table scan instead, confirming that index design must match query patterns.

3.4 : Query Execution – VDBE Bytecode

SQLite is unique among database systems because it compiles SQL into an intermediate bytecode representation before execution, similar to how Java compiles to JVM bytecode before running on the Java Virtual Machine. MySQL and PostgreSQL do not use this virtual machine approach, they execute queries more directly without an intermediate bytecode layer. The advantage of SQLite's VDBE approach is portability and transparency, the bytecode can be inspected directly using the EXPLAIN command, which this project used to provide machine-level proof of how B-Tree indexing changes query execution internally. Without an index the bytecode shows a full table scan loop iterating through all 1 million rows. With an index that loop is completely eliminated and replaced by a direct jump to matching entries, proving that B-Tree indexing doesn't just make queries faster, it fundamentally changes how SQLite executes them at the machine instruction level.

Application design

4.1 : System Overview

The application is a Flight Search System implemented as a Jupyter Notebook using Python and SQLite. The system stores 1 million synthetic flight records and provides a user-facing interface for searching, filtering, and sorting flights. The application was intentionally designed to be simple so that focus could be placed on demonstrating internal database behavior rather than user interface complexity.

4.2 : Database Schema

The database consists of a single table called flights with the following schema:

```
CREATE TABLE IF NOT EXISTS flights (
    id INTEGER PRIMARY KEY,
    airline     TEXT,
    origin      TEXT,
    destination TEXT,
    departure   TEXT,
    price       REAL
)
```

The schema was kept intentionally simple, a single table storing all flight records, to ensure that all performance differences observed are attributable to indexing behavior and not to query

4.3 : Dataset

The dataset consists of 1 million synthetic flight records generated using Python's random module. Records include 5 real US airlines (Delta, United, American, Southwest, JetBlue) and 8 real US airport codes (LAX, JFK, ORD, ATL, SFO, DFW, MIA, SEA). Departure times are distributed across June 2025, and prices range randomly between $80 and $800. The dataset was scaled from 100,000 rows to 1 million rows during experimentation to demonstrate how performance differences become more dramatic at scale.

4.4 : Application Operations

The application supports the following operations:

- Search flights by route (origin + destination) - primary operation
- Filter flights by airline -single column index experiment
- Sort flights by price - ORDER BY optimization experiment
- Insert flight records - read-write tradeoff experiment
- Interactive UI with live DB strategy display

4.5 : User Interface

The user interface is implemented using ipywidgets inside Jupyter Notebook. It provides dropdowns for departure airport, arrival airport, airline filter, and sort preference, along with a Search button. The results display includes the number of matching flights, query execution time, and most importantly, the DB strategy indicator, which shows in real time whether SQLite used an INDEX SCAN (B-Tree) or FULL TABLE SCAN. This directly connects the user experience to the database internals demonstrated throughout the project.

5) Mapping Internals to Application Behavior

5.1 : Operation 1-Search by Route

What the application does:

The user selects a departure and arrival airport. The application runs: SELECT * FROM flights WHERE origin='LAX' AND destination='JFK'

What SQLite does internally:

Without the composite index, SQLite performs a full table scan, reading all 1 million rows and checking each for matching origin and destination. EXPLAIN QUERY PLAN confirms this with output: SCAN flights. With the composite index on (origin, destination), SQLite's query planner selects an index scan, navigating the B-Tree directly to the LAX section then to the JFK entries. EXPLAIN QUERY PLAN confirms this with output: SEARCH flights USING INDEX idx_origin_destination. VDBE bytecode confirms the difference at the machine level, Rewind and Next loop replaced by SeekGE direct jump.

Why it matters:

Query time dropped from 0.54 seconds to 0.005 seconds, a 100x improvement. In a real airline system processing thousands of simultaneous user searches, this difference is the boundary between a usable application and one that times out under load.

5.2 Operation 2 - Filter by Airline

What the application does:

The user selects Delta from the airline dropdown. The application adds AND airline='Delta' to the query dynamically.

What SQLite does internally:

With a single-column index on airline, SQLite navigates the B-Tree to the Delta entries. However, airline has only 5 distinct values, so approximately 200,000 rows still match. The index helps but the result set remains large.

Why it matters:

Only 11x speedup versus 90-100x for the composite index. This demonstrates index selectivity, the more unique values a column has, the more powerful the index. This is why composite indexes on specific combinations outperform single-column indexes on low-cardinality columns.

5.3 Operation 3 - Sort by Price

What the application does:

The user selects Price Low to High. The application runs: SELECT * FROM flights WHERE origin='LAX' ORDER BY price ASC LIMIT 10

What SQLite does internally:

Without a price index, SQLite must find all LAX flights, load them all into memory, sort them by price, then return the top 10 two expensive operations. With a price index, the B-Tree is already stored in sorted order. SQLite walks the tree from cheapest to most expensive and grabs the first 10 matching entries, no separate sorting step needed.

Why it matters:

630x speedup, the most dramatic result in the project. This proves indexes don't just help with filtering, they also eliminate the computational cost of sorting, making ORDER BY queries dramatically faster on indexed columns.

5.4 Operation 4 - INSERT (Write Performance)

What the application does:

New flight records are inserted into the database using INSERT statements.

What SQLite does internally:

Without indexes, SQLite updates only one B-Tree, the main flights table. With two indexes (composite on origin+destination and single-column on airline), SQLite must update three B-Trees simultaneously for every INSERT, the main table plus both index B-Trees, while maintaining sorted order throughout each structure.

Why it matters:

Inserts became 2.7x slower with indexes. This is the fundamental read-write tradeoff of B-Tree indexing, faster reads always come at the cost of slower writes. This is why database engineers carefully choose which columns to index rather than indexing everything.

5.5 Operation 5 - Wrong Index Detection

What the application does:

The application searches by origin and destination even though only a price index exists.

What SQLite does internally:

SQLite's query planner evaluates the price index and determines it has no information about airport codes. The planner ignores the price index entirely and falls back to a full table scan. EXPLAIN QUERY PLAN confirms this, showing SCAN flights even though an index exists.

Why it matters:

This proves that SQLite's query planner is intelligent , it only uses indexes that actually help. It also proves that having an index is not enough, we need the right index on the right columns matching our query patterns.

5.6 Complete Results Summary

Experiment	Without Index	With Index	Speedup
Route search (LAX→JFK)	0.5394s	0.0058s	~92x faster
Route search (ORD→MIA)	0.5456s	0.0052s	~104x faster
Route search (ATL→SFO)	0.4801s	0.0053s	~90x faster
Route search (DFW→SEA)	0.4899s	0.0054s	~91x faster
Airline filter (Delta)	~1.06s	~0.09s	~11x faster
Sort by price (LAX)	~1.07s	~0.002s	~630x faster
INSERT 10,000 rows	0.0140s	0.0372s	2.7x slower

6) Comparison with MYSQL & MongoDB

6.1 : SQLite vs MySQL

MySQL is a full client-server relational database system. Unlike SQLite, MySQL requires a separate server process to be running at all times, manages connection pooling for concurrent users, and stores data in a more complex file format. MySQL also uses B-Tree indexing through its InnoDB storage engine, but with important differences. MySQL's B-Tree indexes use clustered indexing, the primary key index stores the actual row data in the leaf nodes rather than just pointers. This means primary key lookups are marginally faster in MySQL, but secondary index lookups require an additional step to fetch the full row from the primary index.

SQLite's B-Tree implementation uses non-clustered indexing for secondary indexes, the index B-Tree stores index values plus row ID pointers, and a separate lookup is required to fetch the full row from the main table. This is visible in the VDBE bytecode as the DeferredSeek instruction. For the purpose of this project, studying indexing behavior on a single-user local dataset, SQLite's architecture is more transparent and easier to analyze than MySQL's more complex multi-threaded execution model.

Feature	SQLite	MySQL
Architecture	Embedded, serverless	Client-server
Storage	Single .db file	Multiple files + InnoDB tablespace
Index type	Non-clustered B-Tree	Clustered B-Tree (InnoDB)
Concurrent users	Single writer	Multiple concurrent writers
EXPLAIN tool	EXPLAIN QUERY PLAN + VDBE	EXPLAIN + EXPLAIN ANALYZE
Best for	Single-app, analysis, embedded	Web apps, concurrent workloads

6.2 : SQLite vs MongoDB

MongoDB is a NoSQL document database that stores data as JSON-like BSON documents rather than rows in tables. MongoDB does not use B-Tree indexing in the same way as SQLite , it uses a WiredTiger storage engine that supports B-Tree indexes on document fields but also supports more complex index types including compound indexes, text indexes, geospatial indexes, and partial indexes. MongoDB's query planner is significantly more sophisticated than SQLite's, using cost-based optimization with detailed statistics about data distribution.

A key difference relevant to this project is that MongoDB does not have a direct equivalent to SQLite's EXPLAIN QUERY PLAN or VDBE bytecode. MongoDB's explain() method provides query plan information but at a higher level of abstraction. This makes SQLite significantly more suitable for studying the relationship between indexing and query execution at the internal level, which is the core goal of this project.

MongoDB's document model also means that queries filtering on nested fields behave differently from relational queries a flight search in MongoDB would filter on document fields rather than table columns, and index creation syntax differs accordingly. For a structured, tabular dataset like flight records with fixed schema, SQLite's relational model is a more natural and efficient fit.

Feature	SQLite	MongoDB
Data model	Relational — rows and columns	Document — JSON/BSON
Index type	B-Tree	B-Tree, text, geo, partial
Schema	Fixed schema	Flexible, schema-less
Query language	SQL	MongoDB Query Language (MQL)
Internal analysis	EXPLAIN QUERY PLAN + VDBE	explain() — less detailed
Best for	Structured data, analysis	Flexible documents, unstructured data

7) Limitations and Lessons Learned

7.1 Limitations

- **Synthetic dataset**- All 1 million flight records were randomly generated. A real-world dataset from an airline system might have different data distributions, more columns, and more complex relationships that could affect indexing behavior differently.
- **Single-user environment**- SQLite is designed for single-writer access. A real flight search system would need to handle thousands of concurrent users simultaneously, which is where MySQL or PostgreSQL with connection pooling would be necessary.
- **Single table schema**- The project uses only one table to keep the focus on indexing. A production system would have multiple related tables requiring join optimization in addition to single-table index design.
- **Timing variability**- Query times showed slight variation between runs due to CPU load, memory caching, and the overhead of dropping and recreating indexes between tests. Results were consistent enough to demonstrate the pattern but not perfectly reproducible to the millisecond.
- **No distribution analysis**- SQLite does not support distributed database features. The project could not explore topics like sharding, replication, or partition-level index optimization that would be relevant in larger systems like CockroachDB or Cassandra.

7.2 Lessons Learned

- **Index design matters more than just having an index**- the wrong index is as bad as no index. This was demonstrated by the wrong index experiment where a price index was completely ignored by the query planner for an origin-destination query.
- **Index selectivity is the key metric**- a composite index on two specific columns dramatically outperformed a single-column index on a low-cardinality column. Understanding selectivity is essential for designing effective indexes.
- **Indexes are not free**- the 2.7x write slowdown from the INSERT tradeoff experiment demonstrates that every index added to a table increases the cost of write operations. Index design requires balancing read performance gains against write overhead.
- **Internal tools like EXPLAIN QUERY PLAN and VDBE bytecode are invaluable**- being able to see exactly what the database is doing internally, at both the query plan level and the machine instruction level, provides a depth of understanding that timing measurements alone cannot give.
- **Scale reveals what small datasets hide** - at 100,000 rows the performance difference was noticeable but modest. At 1 million rows the same index gave 90-100x improvement, demonstrating why index design decisions that seem minor in development become critical in production.

Git Repo: https://github.com/abhey-10/DSCI551-Flight-Search-SQLite